

CO321 Embedded Systems - 2023

Lab 1 - Programming AVR Microcontrollers

Programming ATmega328p in C language

Arduino IDE provides an abstract way to program the microcontroller easily and efficiently. In this lab we manually do what is happening inside the Arduino IDE, which is hidden to the user.

In our particular case, we use the **avr-gcc** compiler and the **avrdude** uploading tool. We can use these tools to write a program in *C language*, *instead of the Arduino language*, build/compile, and upload that program to the AVR. We build our *C* program by calling the **avr-gcc** command and upload it using **avrdude** command.

avr-gcc toolchain doesn't know the Arduino Uno board layout, it only knows the microcontroller that is mounted on it, which is the **ATmega328p**. So, we need to use the *hardware I/O registers* and *pins* of that particular chip when writing our program. However, we have to use the pins on the Arduino board for connecting to external components (because the chip is already soldered to the board). Figure 1 shows the mapping between the pins of the microcontroller and the pins of the Arduino board.

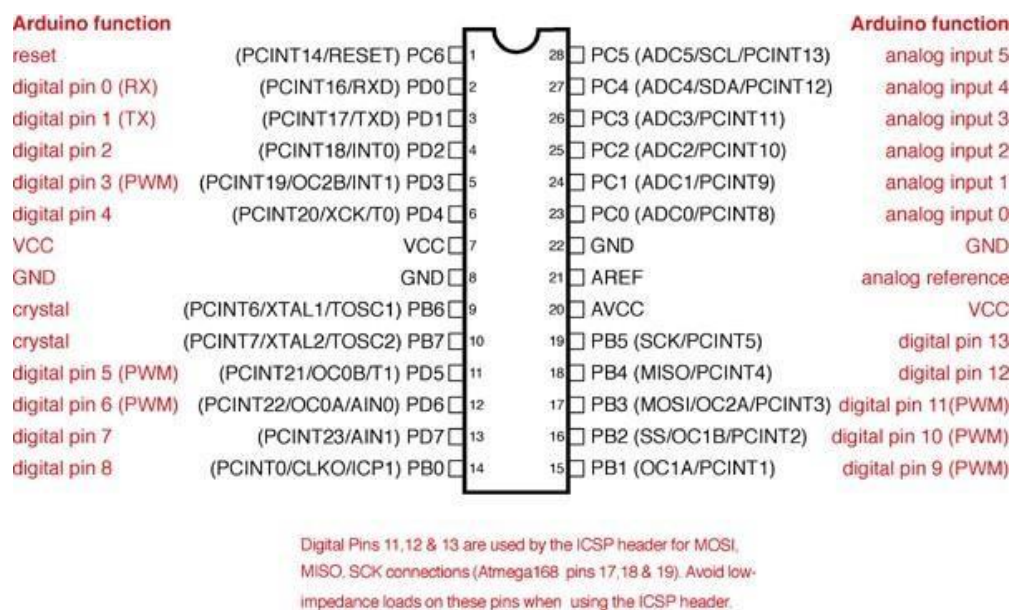


Figure 1 : Pin mapping of ATmega328P to Arduino Uno

VCC is the pin which provides power to the microcontroller and GND pins are the grounds. There are three ports namely *Port B (PB)*, *Port C (PC)* and *Port D (PD)*. PB and PD are 8-bit ports and PC is a 7-bit port. The ports are bi-directional, and they can be used for both input

and output. Some pins have multiple functionalities as well where the alternate functionalities are shown in brackets. For example, *PC6* which is the 7th bit of *Port C* can be used as RESET input, as shown in figure 1. These different pins in the microcontroller are connected appropriately inside the Arduino and the ports are exposed to the outside by different names. For example, the *PB5* pin of the ATmega328p is exposed by the name “digital pin 13” on the Arduino.

Blinking LED Example:

Let’s start with an example: An LED blinking on Arduino digital pin 13.

Digital pin 13 is mapped to the *PB5* pin of the ATmega328p chip and hence the I/O register for that pin (*PORTB*) must be programmed.

- Create a folder for your project and write the following example program in a file called “**led.c**”.

```
#include <avr/io.h>
#include <util/delay.h>

#define BLINK_DELAY_MS

1000int main (void){

  DDRB = DDRB|(1<<5);    /* configure pin 5 of PORTB for output*/

  while(1){

    PORTB = PORTB | (1<<5);    /* set pin 5 high to turn led on */

    _delay_ms(BLINK_DELAY_MS);

    PORTB = PORTB & ~(1<<5);    /* set pin 5 low to turn led off */

    _delay_ms(BLINK_DELAY_MS);

  }
}
```

DDRB is the **Data Direction Register** for *Port B*. Since we are using this as an output, bit 5 of this register must be set to 1. First we read the current value in *DDRB* and then apply bitwise or with 00100000 and write back to *DDRB*. This preserves the state of other bits and change only the 5th bit.

Then inside the loop we turn on the LED and turn off while keeping enough delays in-between. The *_delay_ms* function delays for the number of milliseconds provided as an argument. *PORTB = PORTB | (1<<5)* sets the 5th bit of *Port B* register to 1, and *PORTB = PORTB & ~(1<<5)* sets the same bit to 0 while preserving the values which were there for other bits. If you don’t want to preserve the other bits, you can just say *PORTB=0xFF* to turn “on” all bits in *Port B*, or say *PORTB=0x00* to turn “off” all bits

.

Now it's time to run the commands that build and upload the LED blink program.

The commands that we used to compile and upload the file "led.c" are:

```
$ avr-gcc -Os -DF_CPU=16000000UL -mmcu=atmega328p -o led led.c
$ avr-objcopy -O ihex -R .eeprom led led.hex
$ avrdude -F -V -c arduino -p ATMEGA328P -P /dev/ttyACM0 -b 115200 -U
flash:w:led.hex

avrdude: AVR device initialized and ready to accept instructions

Reading | ##### | 100% 0.00s

avrdude: Device signature = 0x1e950f
avrdude: NOTE: FLASH memory has been specified, an erase cycle will be
performedTo disable this feature, specify the -D option.
avrdude: erasing chip
avrdude: reading input file "led.hex"
avrdude: input file led.hex auto detected as Intel Hex
avrdude: writing flash (88 bytes):

Writing | ##### | 100% 0.02s

avrdude: 88 bytes of flash written
avrdude: safemode: Fuses OK
avrdude done. Thank you.
```

The **avr-gcc** compiler creates an executable binary program that contains machine code. We need to create a HEX file and then write it to the Program Memory inside the microcontroller. Binary is converted to HEX using **avr-objcopy**. The Arduino board provides an interface to the microcontroller as a virtual Serial port and we are uploading the hex file to the microcontroller via this. Uploading is done by the command **avrdude**.

The explanation of the commands are as follows.

- The first command takes the C source file and compiles it into a binary file. The options tell the compiler to optimize for code size, what the clock frequency is (16 MHz for Arduino Uno) and which particular microcontroller we're using.
- The second command converts the binary program into a HEX file format.
- The fourth command uploads the HEX data into the ATMEGA chip embedded flash, and the options tells avrdude tool to communicate using the Arduino serial protocol, through a particular serial port and to use 115200bps as the data rate.

After this is done, you should see the LED blinking.

Lab Exercises:

- Connect four (4) LEDs to *Port B* and blink all of them simultaneously.
- Using four (4) LEDs, implement a "Knight Rider" style display.
If the four LEDs are *A*, *B*, *C* and *D* then the turn order of lights should be: ABCDCBABC.....

Submit a .zip file named "groupXX.zip" containing all your code files and a screenshot/photo of the circuit.

Post-lab notes:

Some might want to try out this method at home on your own Arduino device. If you are using Linux Ubuntu, simply install the following three packages via `apt -get install`.

- `gcc-avr`
- `avr-libc`
- `avrdude`

Lab 2 - Interrupts

AVR Interrupt Programming in C

Introduction

Interrupts vs. Polling

A Single microcontroller can serve several peripherals. There are two methods by which devices receive service from the microcontroller: *interrupts and polling*.

In the interrupt method, whenever any peripheral needs the microcontroller's service, the device notifies it by sending an interrupt signal. Upon receiving an interrupt signal, the microcontroller stops whatever it is doing and serves the device. The program associated with the interrupt is called the *Interrupt Service Routine (ISR)*.

In polling, the microcontroller continuously monitors the status of several devices and serve each of them as certain conditions are met. However, it is not an efficient use of the microcontroller.

The most important reason why the interrupt method is preferable is that; the polling method wastes much of the microcontroller's time by *polling devices that do not need service*.

Interrupt Service Routine

For every interrupt, there must be an interrupt service routine. When an interrupt is invoked, the microcontroller runs the interrupt service routine.

Steps in executing an ISR upon activation of an interrupt

The microcontroller goes through the following steps:

1. It finishes the instruction that is currently being executed and saves the address of the next instruction on the stack.
2. It jumps to a fixed location in memory called the interrupt vector table. The interrupt vector table directs the Microcontroller to the address of the interrupt service routine (ISR).

3. The Microcontroller starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine, which is RETI (return from interrupt).
4. Upon executing the RETI instruction, the Microcontroller returns to the place where it was interrupted. First, it gets the program counter address from the stack by popping the top bytes of the stack into the PC. Then it starts to execute from that address.

Interrupt	ROM Location (Hex)
Reset	0000
External Interrupt request 0	0002
External Interrupt request 1	0004
External Interrupt request 2	0006
Time/Counter2 Compare Match	0008
Time/Counter2 Overflow	000A
Time/Counter1 Capture Event	000C
Time/Counter1 Compare Match A	000E
Time/Counter1 Compare Match B	0010
Time/Counter1 Overflow	0012
Time/Counter0 Compare Match	0014
Time/Counter0 Overflow	0016
SPI Transfer complete	0018
USART, Receive complete	001A
USART, Data Register Empty	001C
USART, Transmit Complete	001E
ADC Conversion complete	0020
EEPROM ready	0022
Analog Comparator	0024
Two-wire Serial Interface (I2C)	0026
Store Program Memory Ready	0028

Figure 1: Example Interrupt Vector Table

Sources of interrupts in the AVR

There are many sources of interrupts in the AVR, depending on which peripheral is incorporated into the chip. The following are some of the most widely used sources of interrupts in the AVR.

1. There are at least two interrupts set aside for each of the timers, one for overflow and another for comparing match.
2. Three interrupts are set aside for external hardware interrupts. Pins PORTD.2, PORTD.3 and PORTB.2 are for the external hardware interrupts INTO, INT1, and INT2, respectively.
3. Serial communication's USART has three interrupts, one for receiving and two interrupts for transmitting.
4. The SPI interrupts.
5. The ADC (analog-to-digital converter). Normally, the service routine for an interrupt is too long to fit into the memory space allocated. For that reason, a JMP instruction is placed on the vector table to point to the address of the ISR.

Note: In this lab, we will only focus on external interrupts

Enabling and disabling an interrupt

Upon reset, all interrupts are disabled, meaning that none will be responded to by the microcontroller unless they are activated. The 7th bit for the SREG (Status Register) register is responsible for enabling and disabling the interrupts globally.



Figure 2: Status Register

Steps in enabling an interrupt

1. Bit 7(I) of the SREG register must be set to HIGH to allow the interrupts to happen.
2. If I=1, the corresponding interrupt enable (IE) flag bit for the required interrupt each interrupt must be enabled by setting to HIGH. This flag bit is usually located in a register belonging to the corresponding peripheral.

Interrupt Priority

If two interrupts are activated at the same time, the interrupt with the higher priority is served first. The priority of each interrupt is related to the address of that interrupt in the interrupt vector. The interrupt that has a lower address, has a higher priority.

Interrupt inside an interrupt

When the AVR begins to execute an ISR, it disables the I bit of the SREG register, causing all the interrupts to be disabled, and no other interrupt occurs while serving the interrupt. After serving the interrupt, the AVR enables the I bit, causing the other interrupts to be served.

Programming External Hardware Interrupts

The ATmega has two external hardware interrupts: pins PD2 and PD3 designated as INTO, and INT1 respectively. Upon activation of these pins, the AVR is interrupted in whatever it is doing and jumps to the vector table to perform the interrupt service routine.

The hardware interrupts must be enabled before they can take effect. This is done using the INTx bit located in the EIMSK (External Interrupt Mask Register) register.

ISC11, ISC10, ISCo1 and ISCo0 bits in the EICRA (External Interrupt Control Register A) register defines which activity on the corresponding pin will generate an interrupt as shown in figure below.

D7				D0			
SE	SM2	SM1	SM0	ISC11	ISC10	ISC01	ISC00

ISC01, ISC00 (Interrupt Sense Control bits) These bits define the level or edge on the external INT0 pin that activates the interrupt, as shown in the following table:

ISC01	ISC00		Description
0	0		The low level of INT0 generates an interrupt request.
0	1		Any logical change on INT0 generates an interrupt request.
1	0		The falling edge of INT0 generates an interrupt request.
1	1		The rising edge of INT0 generates an interrupt request.

ISC11, ISC10 These bits define the level or edge that activates the INT1 pin.

ISC11	ISC10		Description
0	0		The low level of INT1 generates an interrupt request.
0	1		Any logical change on INT1 generates an interrupt request.
1	0		The falling edge of INT1 generates an interrupt request.

Interrupt Programming in C

Interrupt include file:

We should include the interrupt header file if we want to use interrupts in our program. Use the following statement:

```
#include <avr/interrupt.h>
```

cli() and sei():

cli() macro clears the I bit of the SREG register while sei() sets it.

Defining an ISR:

To write an ISR for an interrupt we use the following structure:

```
ISR (interrupt vector name) {
    //your program
}
```

For the interrupt vector name we must use the ISR names in below table. For example, the following ISR serves the EEPROM ready interrupt:

```
ISR (EE_RDY_vect) {...}
```


Interrupt	Vector Name in WinAVR
External Interrupt request 0	INT0_vect
External Interrupt request 1	INT1_vect
External Interrupt request 2	INT2_vect
Time/Counter2 Compare Match	TIMER2_COMP_vect
Time/Counter2 Overflow	TIMER2_OVF_vect
Time/Counter1 Capture Event	TIMER1_CAPT_vect
Time/Counter1 Compare Match A	TIMER1_COMPA_vect
Time/Counter1 Compare Match B	TIMER1_COMPB_vect
Time/Counter1 Overflow	TIMER1_OVF_vect
Time/Counter0 Compare Match	TIMER0_COMP_vect
Time/Counter0 Overflow	TIMER0_OVF_vect
SPI Transfer complete	SPI_STC_vect
USART, Receive complete	USART0_RX_vect
USART, Data Register Empty	USART0_UDRE_vect
USART, Transmit Complete	USART0_TX_vect
ADC Conversion complete	ADC_vect
EEPROM ready	EE_RDY_vect
Analog Comparator	ANALOG_COMP_vect
Two-wire Serial Interface	TWI_vect
Store Program Memory Ready	SPM_RDY_vect

Example:

Using INT0 to generate a program to switch on a LED connected to PB6 when a button is pressed. (The LED is switched on the rising edge, i.e. when the button is pressed)

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

#define BLINK_DELAY_MS 100

int main() {
    DDRD &= ~(1<<2); //PD2 (INT0) is input

    DDRB |= (1<<0); //PB0 is output

    EICRA |= (1<<ISC01); //set for rising edge detection
    EICRA |= (1<<ISC00); //set for rising edge detection

    sei(); //enable global interrupts

    EIMSK |= (1<<INT0); //enable external interrupt for int0

    while(1) {
    }
    return 0;
}

ISR(INT0_vect) {
    PORTB &= ~(1<<0);
    _delay_ms(BLINK_DELAY_MS);
    PORTB |= (1<<0);
}
```

Exercises:

1. PD7 pin is connected to a push button. Write a program that uses 6 LEDs connected to PORTB (6 LSBs) to display the number of times the push button is pressed, as a binary number. Implement **without using external interrupts**. (i.e. Use polling)
2. Write a program that toggles pin PBo (show using a connected LED) whenever the push button is released, **using external interrupts**.
3. Extend the program in part 2, so that the number of times the push button was released is displayed as a binary number on a set of LEDs connected to PORTB.
4. Connect two push buttons (A and B) to two external interrupt pins. Connect 6 LEDs to a GPIO port. Write a program where a user can input a binary number using the two push buttons (press A to input a '0', press B to input a '1') through external interrupts, and display that number using the LEDs. The number must be updated and displayed every time one of the buttons are pressed.

Lab 3 - Timers

Using Timers in AVR

Introduction to timers/counters

Many applications need to count occurrences of an event or generate time delays. So, there are counter registers in microcontrollers for this purpose. When we want to *count events*, we connect the external event source to the clock pin of the counter register. Then, when an event occurs externally, the content of the counter is incremented; this way, the content of the counter represents *how many times the event has occurred*.

When we want to generate time delays, we *connect the oscillator* to the clock pin of the counter. So, when the oscillator 'ticks', the content of the counter is incremented. As a result, the content of the counter register represents how many 'tick' events have occurred from the time we have cleared the counter. Since the speed of the oscillator in a microcontroller is known, we can calculate the 'tick' period. Therefore, using the content of the counter register we can derive how much time has elapsed altogether.

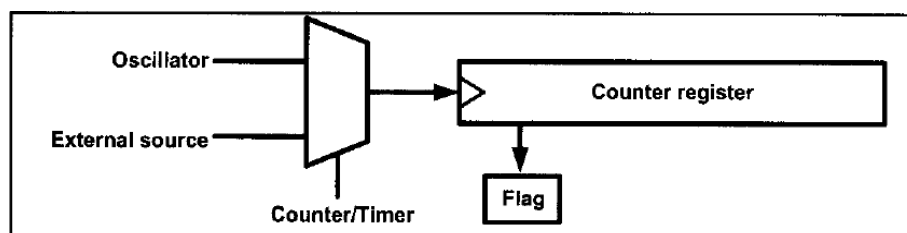


Figure 1: A high-level schematic of counters and timers in microcontrollers

Figure 1 shows the high-level schematic of a timer module. All the Atmel microcontrollers have Timers as an inbuilt peripheral. ATmega328P has three timers: *TIMER0*, *TIMER1*, and *TIMER2*. They also have a *Watchdog Timer*, which can be used as a safeguard or software reset mechanism.

Here are a few details about each timer:

TIMER0

TIMER0 is an 8-bit timer, meaning its counter register can record a maximum value of 255 (unsigned 8-bit). *TIMER0* is used by native Arduino timing functions such as `delay()`.

TIMER1

TIMER1 is a 16-bit timer, with a maximum counter value of 65535 (unsigned 16-bit integer). The Arduino Servo library uses this timer.

TIMER2

TIMER2 is an 8-bit timer that is very similar to *TIMER0*. It is utilized by the Arduino `tone()` function.

Using *TIMER0*

To use a timer, we need to set it up, and then get it to start counting clock 'ticks'. To do this, we'll use built-in registers on the AVR chip that store timer settings. Each timer involves a number of registers that do various things and the registers corresponding to *TIMER0* are described below. When programming, we can access these timer registers directly using their names.

• **TCCR0A and TCCR0B**

These two 8-bit registers hold setup values for the *TIMER0*. TCCR stands for *Timer/Counter Control Register*.

Each register holds 8 bits, and each bit stores a configuration value as shown in Figure 2.

TCCR0A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR0B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 2: Timer/Counter control registers A & B

To start using our timer, the most important settings are the last three bits in TCCR0B: CS12, CS11, and CS10. These dictate the timer clock signal (external or oscillator) and prescaling. By setting these bits in various combinations, we can tell the timer to count at different scales of the oscillator clock speed ($\times 1$, $\times \frac{1}{8}$, $\times \frac{1}{64}$, etc.). In other words, we can slow down the timer's clock compared to the oscillator clock. Figure 3 shows the relevant table from the data sheet.

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk _{I/O} /(No prescaling)
0	1	0	clk _{I/O} /8 (From prescaler)
0	1	1	clk _{I/O} /64 (From prescaler)
1	0	0	clk _{I/O} /256 (From prescaler)
1	0	1	clk _{I/O} /1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Figure 3: Clock select bit description

• TCNT0

This is the *Timer/Counter Register* where the 8-bit counter value for *TIMER0* resides. The value of the counter is stored here and increases/decreases automatically. Data can be both read/written from this register.

• TIFR0

The *Timer/Counter Interrupt Flag Register* is a register shared by all the timers. Bits 1 and 0 are allotted for *TIMER0*. At present we are interested in the zeroth bit named as TOV0 bit. This bit is set ('1') whenever *TIMER0*'s counter *overflows* (exceeds the maximum value). This bit is cleared ('0') whenever the corresponding Interrupt Service Routine (ISR) is executed. If there is no ISR in the program to execute, we should clear it manually by writing logic '1' to it.

For complete information about the registers, refer to section 15.9 in the ATmega328P datasheet.

Modes of operation for *TIMER0*

TIMER0 has several modes of operation, that is, different behaviors for the Timer/Counter. They are namely, Normal Mode, Clear Timer on Compare Match (CTC) Mode, Fast PWM Mode and Phase Correct PWM Mode. The mode is defined by the combination of the Waveform Generation mode (WGM02:0) and Compare Output mode (COM0x1:0) bits. In this class, we are only focusing on the normal mode.

Normal mode of *TIMER0*

In this mode, the value of the timer/counter increments with the clock. It counts up until it reaches its max of 0xFF. When it rolls over from 0xFF to 0x00, it sets the flag bit called TOV0 (Time Overflow) in TIFR0 Register. This time flag can be monitored.

For complete information about the modes of operation, refer to section 15.7 in the ATmega328P datasheet.

Steps to program *TIMERO* in normal mode

To generate a time delay using *TIMERO* in Normal mode, the following steps can be taken. (note that this description uses a polling method instead of interrupts).

1. Load the TCNT0 register with the *initial count value*.
2. Load the values into the TCCR0A and TCCR0B register, indicating which mode is to be used and the prescaler option (you will have to refer to the datasheet to identify the bits that should be set for the corresponding mode). When you select the clock source, the timer/counter starts to count, and each tick causes the content of the timer/counter to increment by 1.
3. Keep monitoring the Timer Overflow flag (TOV0) to see if it is raised. Get-out of the loop when TOV0 becomes high.
4. Stop the timer by disconnecting the clock source.
5. Clear the TOV0 flag for the next round.
6. Go back to step 1 to load TCNT0 again.

Example 1:

Write a C program to toggle all the bits of PORTB continuously with a fixed delay. Use *TIMERO*, in normal mode, and no pre-scalar options to generate the delay. (go to next page for the sample code)

Code :

```
#include <avr/io.h>
void delay_timer0(){

    TCNT0 = 0x00;    /*Load timer counter register with 0*/

    TCCR0A = 0x00;    /*Set the Timer0 under normal mode with no prescaler*/
    TCCR0B = 0x01;

    while((TIFR0&0x01)==0); /*Wait till timer overflow bit (TOV0) is set*/

    TCCR0A = 0x00;    /*clear timer settings (this stops the timer)*/
    TCCR0B = 0x00;

    TIFR0 = 0x01;    /*Clear the timer overflow bit (TOV0) for next round*/
    /*strange thing about this flag is that in order to clear it we should
    write 1 to it This rule applies to all flags of AVR chip*/

}

int main (void)
{

    DDRB = DDRB | (1<<5);    /* configure pin 5 of PORTB for output*/

    while(1) {

        PORTB = PORTB | (1<<5);    /* set pin 5 high to turn led on */
        delay_timer0();

        PORTB = PORTB & ~(1<<5);    /* set pin 5 low to turn led off */
        delay_timer0();

    }
}
```

Calculating delay length for timers

The delay length for TIMERO in normal mode depends primarily on two factors:

- a) The crystal oscillator's frequency
- b) The pre-scalar factor

A third factor in the delay size is the C compiler because various C compilers generate different hex code sizes, and amount of overhead due to the instructions varies by compiler.

Example 2:

Write a C program to toggle only bit 5 in PORTB register (PB5) every 70µs. Use *TIMERO*, normal mode, and 1:8 prescaler to create the delay. Assume XTAL=16MHz.

Derivation of initial counter value:

$$\text{XTAL} = 16\text{MHz} \rightarrow T_{\text{xtal_clock}} = \frac{1}{16} \mu\text{s}$$

$$\text{Prescaler} = 1:8 \rightarrow T_{\text{counter_clock}} = 8 \times \frac{1}{16} \mu\text{s} = 0.5 \mu\text{s}$$

$$\text{Counter increments needed} = 70 \mu\text{s} / 0.5 \mu\text{s} = 140 \text{ increments}$$

$$\text{Initial counter value} = 1+255 - 140 = 116$$

Code :

```
#include <avr/io.h>

void delay_timer0(){

    TCNT0 = 116;          /*Load timer counter register*/

    TCCR0A = 0x00;        /*Set the Timer0 under normal mode with 1:8 prescaler*/
    TCCR0B = 0x02;

    while((TIFR0&0x01)==0); /*Wait till timer overflow bit (TOV0) is set*/

    TCCR0A = 0x00;        /*clear timer settings (this stops the timer)*/
    TCCR0B = 0x00;

    TIFR0 = 0x01;         /*Clear the timer overflow bit (TOV0) for next round*/
    /*strange thing about this flag is that inorder to clear it we should write 1 to it.
    This rule applies to all flags of AVR chip*/
}

int main (void)
{

    DDRB = DDRB | (1<<5);      /* configure pin 5 of PORTB for output*/

    while(1) {
        PORTB = PORTB ^ (1<<5); /* toggle pin 5 */
        delay_timer0();
    }
}
```

Exercise 1 :

Write a C program to toggle an LED connected to pin 5 of PORTB register (PB5) every 2ms. Use *TIMERO*, normal mode, and a suitable pre-scalar to create the delay. Assume XTAL= 16 MHz. What is the selected prescaler? What is the initial counter value? Explain the output. What is the reason for it?

Exercise 2 :

Try to increase the delay in Exercise 2 to 500ms. Talk to an instructor and explain how this can be done, or why this cannot be done.

Exercise 3 :

Find out the highest possible countable time interval using *TIMERO*, normal mode, and a suitable pre-scalar. Assume XTAL= 16MHz. What is the selected prescaler? What is the highest countable time interval?

(Complete and show exercises 1 to 3 on the first day of Lab3. Remaining exercises should be finished before coming to the second day of Lab3)

Exercise 4:

Write a program that toggles an LED connected to PB5 pin every second using *TIMER1*, while at the same time operating a Knight Rider style circuit of 4 LEDs (from Lab 1). For the LED toggle you must use the timer overflow interrupts. For the Knight Rider style circuit you are allowed use either timer interrupts, polling, software delays or any other method.

Exercise 5:

You found out the maximum interval you can implement using *TIMERO* in exercise 4. However, by using an 8-bit counter variable and interrupts, you can make this interval much longer. Develop a program to blink an LED with a 100ms interval using *TIMERO* .

Exercise 6:

Implement a program to blink two LEDs. One led should blink with an interval of 50 ms and the other should blink with an interval of 500ms. You must use two timers, and timer overflow interrupts.

Lab 4 - Pulse Width Modulation (PWM)

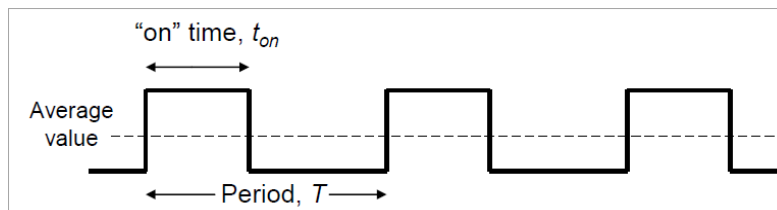
AVR PWM Programming in C Language

Pulse Width Modulation (PWM)

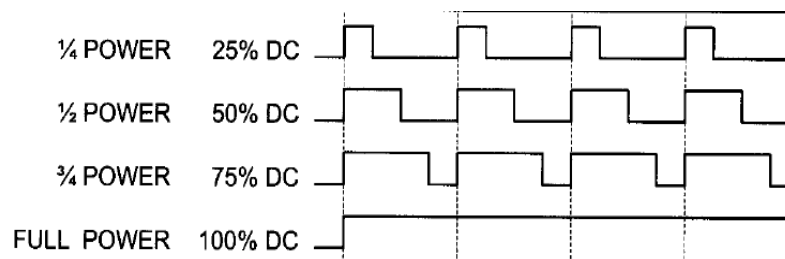
Pulse width modulation (PWM) is a powerful technique for controlling analog circuits with a microprocessor's digital output. It is a simple method of using a rectangular digital waveform to control an analog variable. PWM control is used in a variety of applications, ranging from communications to automatic control.

Period T is normally kept constant. Pulse width, or “on” time, is varied to get the desired output. The Duty Cycle is the proportion of time that the pulse is „ON” or „high” , and is expressed as a percentage:

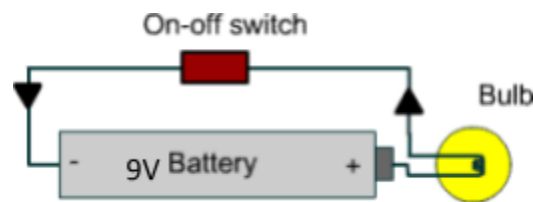
$$\text{Duty cycle} = (\text{pulse ON time} / \text{pulse period } T) * 100\%$$



Whatever duty cycle a PWM stream has, there is an average output voltage, as indicated by the dotted line. If the “on” time is small, the average output is low; if the “on” time is large, the average output is high. **By controlling the duty cycle, we control this average output voltage.**



The figure below shows a simple circuit that could be driven using PWM. In the figure, a 9V battery power an incandescent light bulb.



If we closed the switch connecting the battery and lamp for 50ms, the bulb would receive 9V during that interval and light up. If we then opened the switch for the next 50ms, the bulb would receive 0V. If we repeat this cycle 10 times (a full second), the bulb would appear to be lit as though it were connected to a 4.5V battery (50% of 9V). We say that the duty cycle is 50% and the modulating frequency is 10Hz.

Most loads require a much higher modulating frequency than 10Hz. Imagine that our bulb was switched on for five seconds, then off for five seconds, then on again. The duty cycle would still be 50%, but the bulb would appear brightly lit for the first five seconds and off for the next. In order for the bulb to see an average voltage of 4.5 volts, the cycle period must be short relative to the load's response time to a change with the switch state. To achieve the desired effect of a dimmer-lamp (but always lit), it is necessary to increase the modulating frequency. The same is true in other applications of PWM. Common modulating frequencies range from 1kHz to 200kHz.

PWM with Timero in ATmega 328P

The ATmega328P comes with 3 timers as we discussed. All these timers can be used for PWM. They support two PWM modes; namely, **Fast PWM** and **Phase Correct PWM**. In this lab we will focus on the Fast PWM mode with `TIMER0`. The advantage of using the built-in PWM feature of the AVR (rather than creating PWM in software) is that it gives us the option of pre-programming the period and duty cycle, therefore relieving the CPU to do other important things.

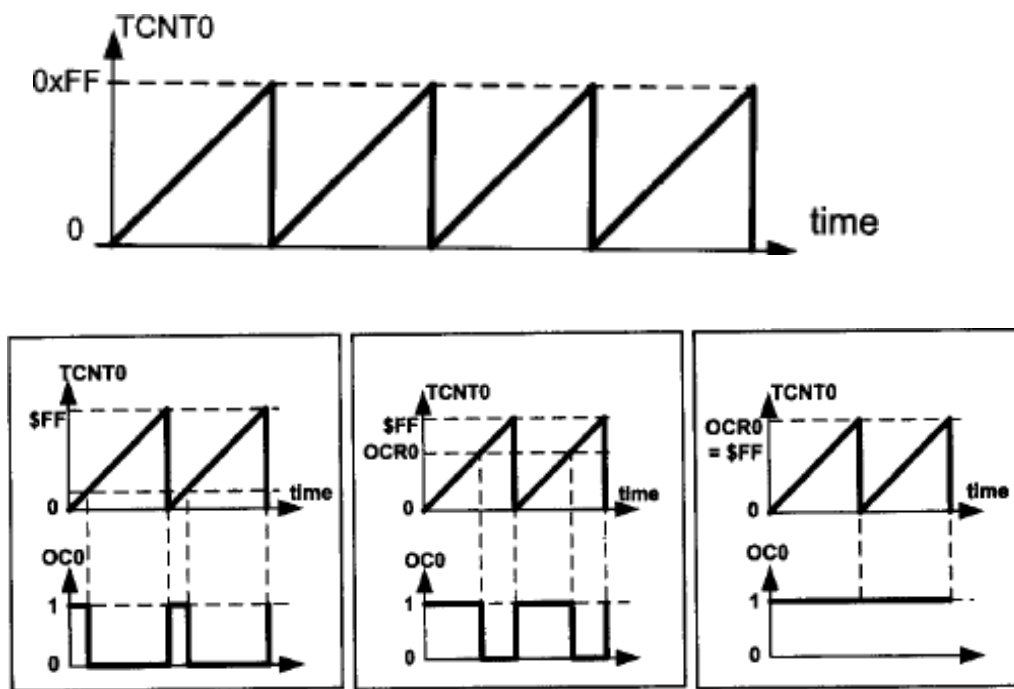
ATmega328 `TIMER0` has 2 outputs, `OC0A` (Timer/Counter 0 Output Compare Match A Output) and `OC0B` (Timer/Counter 0 Output Compare Match B Output). Since both of these outputs run off the same timer and waveform generators both `OC0A` and `OC0B` are synchronized. We will use `OC0A` in this lab.

Registers involved

- TCNT0 – Timer/Counter Register
- OCR0A – Output Compare Register A
- TCCR0A – Timer/Counter Control Register A
- TCCR0B – Timer/Counter Control Register B

In the fast PWM, the counter (TCNT0) counts like it does in the Normal mode. After the time is started, it starts to count up. It counts up until it reaches its limit of $0xFF$. Then it rolls over from $0xFF$ to $0x00$.

In the below figure, you can see the reaction of the waveform generator when compares match occurs while the timer is in the fast PWM mode.

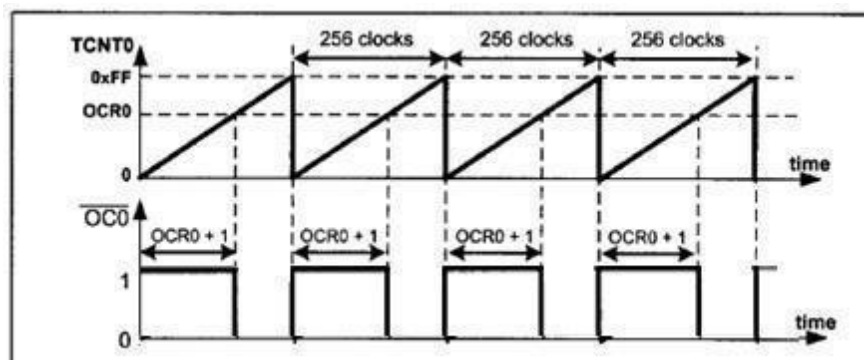


In the figure OC0 is the PWM output pin (Actually, this should be OC0A for our microcontroller). OCR0 is the Output Compare Register (For our microcontroller this should be OCR0A). At the beginning (count 0) PWM pin is set to 1. When the counter value in TCNT0 reaches OCR0 the PWM pin is set to low.

Frequency of the generated wave in Fast PWM mode

The timer counts from 0 to TOP and then rolls over. So, the frequency of the generated wave is 1/256 of the frequency of timer clock. The frequency of the timer clock can be selected using the prescaler. So, in 8-bit timers the frequency of the generated wave can be calculated as follows (N is determined by the prescaler.)

$$\left. \begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{256} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{256 \times N}$$



Duty Cycle of the generated wave in Fast PWM mode

The duty cycle can be determined using the OCR0A register. The bigger OCR0 value results in a bigger duty cycle; When OCR0=255, the OC0 is high 256 clocks out of 256 clocks. The Duty cycle can be calculated using the following formula.

$$\text{Duty Cycle} = \frac{\text{OCR0} + 1}{256} \times 100$$

Note that in ATmega 328P this waveform is called the non-inverted mode. There is another mode called inverted mode where the PWM waveform would be inverted. In this lab we will only deal with the non-inverted mode.

Steps in programming

- Configure the corresponding PWM pin as an output pin.
- Find the counts required for the given duty cycle and write it to `OCR0A`.
- Configure the PWM settings.
 - Select the timer mode. In lab 4 we used the normal mode. But today we should use Fast PWM mode. (See `WGM01:0` bits in `TCCR0A` and `WGM02` in `TCCR0B`)
 - We are using a non inverting mode. So select that. (See `COM0A1` and `COM0A0` bits in `TCCR0A`. Use the correct table for Fast PWM since we are using that)
- Select the clock source. (See `CS02:0` in `TCCR0B`) You should use the correct prescaler to match the required period for the generated PWM. Note that when you select the clock source, the timer starts running and hence the PWM waveform should appear on the corresponding PWM pin.
- Now PWM is running. You can use the CPU for something else. If not, remember to put a `while(1)` loop to prevent the program from ending.

Exercises

1. Write a program that generates a PWM waveform with a frequency of approximately 976.56Hz and a duty cycle of 50%. Connect one LED to the PWM output pin and another to the 5V output pin (LEDs and the resistors you use should be identical) and check if the intensity of the bulb connected to the PWM is half of the other.
2. Now extend the program in part 1 above, to **slowly** fade an LED from off->on and then on->off repeatedly.
3. The frequency of the sound made by a piezo buzzer depends on the frequency of the PWM waveform applied to it. In this case, the *duty cycle* affects the *volume* of the sound. Write a program that repeatedly generates 4 separate sound frequencies in increasing order one after the other. (each frequency should be heard for a notable time before switching to the next frequency so that it can be heard properly). Keep in mind that humans can hear frequencies in the range 20Hz - 20,000Hz only.